QR Watchman Attendance System

> A full-stack QR-based attendance system for security guards and watchmen — with geofence validation, face capture, shift management, and an admin portal.

Table of Contents

- How It Works
- Features
- Tech Stack
- Project Structure
- Prerequisites
- Local Development
- Build
- Deployment
  - Backend — Azure App Service
  - Frontend — Cloudflare Pages
  - Optional — Cloudflare Worker Proxy
  - Frontend — Azure Static Web Apps (alternative)
- Production Checklist
- Troubleshooting

Google Maps API Key Setup

This project uses the Google Maps JavaScript API.
You must provide your own API key.

1. Go to Google Cloud Console
2. Create a Google Maps JavaScript API key
3. Replace the API key in the following file:

```
WatchmanAttendance.UI/src/index.html
```

line:

```
<script src="https://maps.googleapis.com/maps/api/js?key=YOUR_API_KEY&loading=async"></script>
```

How It Works

```
Admin creates QR location
        |
        ▼
Watchman scans QR code (mobile browser)
        |
        ▼
API validates QR token + geofence radius
        |
        ▼
Watchman logs in (if not already authenticated)
        |
        ▼
Watchman captures face photo + GPS position
        |
        ▼
API records attendance log
        |
        ▼
Admin views records, approves requests, audits logs
```

Features

Admin

Feature	Description
Watchman management	Create, view, and manage watchman accounts
QR locations	Generate QR codes pinned to GPS locations with configurable radius
Attendance records	View, filter, and delete attendance logs
Attendance requests	Review and approve/reject correction requests
Shift management	Assign and manage watchman shifts
Audit logs	Full audit trail of admin actions

Watchman

Feature	Description
QR scan	Scan a QR code to start the attendance flow
Geofence check	Location validated against the QR pin radius
Face capture	Take a photo as part of the attendance record
Dashboard	View personal attendance history
Requests	Submit correction requests and track their status

Tech Stack

Layer	Technology
Backend	ASP.NET Core Web API (.NET 8), Dapper, JWT
Database	SQL Server / Azure SQL

Layer	Technology
Frontend	Angular 17, TypeScript, Tailwind CSS
Maps	Google Maps JavaScript API
Hosting (recommended)	Azure App Service + Cloudflare Pages

Project Structure

```
QR-Watchman-Attendance-System/
├── WatchmanAttendance.API/       # ASP.NET Core Web API
│   ├── Controllers/              # API endpoints
│   ├── DTOs/                     # Request/response models
│   ├── Models/                   # Domain models
│   ├── Repositories/             # Data access (Dapper)
│   ├── Services/                 # JWT, geo, timezone helpers
│   ├── appsettings.json          # Configuration
│   └── Program.cs                # App setup, CORS, middleware
│
└── WatchmanAttendance.UI/        # Angular 17 frontend
    └── src/app/
        ├── admin/                # Admin dashboard
        ├── auth/                 # Login pages
        ├── watchman/             # Watchman dashboard, QR scan, attendance
        ├── config/app-config.ts  # API URL config
        └── app.routes.ts         # Routing
```

Prerequisites

- .NET 8 SDK
- Node.js 18+ and npm
- SQL Server or SQL Server Express
- A Google Maps API key (for the map picker)
- *(optional)* Azure account for deployment
- *(optional)* Cloudflare account for Pages / Workers

Local Development

1. Clone the repository

```
git clone https://github.com/your-username/QR-Watchman-Attendance-System.git
cd QR-Watchman-Attendance-System
```

2. Set up the database

Create an empty SQL Server database, then run the setup script:

```
sqlcmd -S localhost -d WatchmanAttendance -i sql/setup.sql
```

Or open sql/setup.sql in SQL Server Management Studio and execute it against your database.

This script creates all tables and inserts the default admin account:

Field	Value
Username	`admin@surge.com`
Password	`12345678`

> **Change the admin password** after your first login in production.

3. Configure the backend

Edit `WatchmanAttendance.API/appsettings.json`:

```json
{
  "ConnectionStrings": {
    "Default": "Server=localhost;Database=WatchmanAttendance;Trusted_Connection=True;"
  },
  "Jwt": {
    "Key": "your-secret-key-at-least-32-characters",
    "Issuer": "WatchmanAttendanceAPI",
    "Audience": "WatchmanAttendanceClient"
  },
  "Frontend": {
    "BaseUrl": "http://localhost:4200"
  }
}
```

> **Never commit real secrets.** Use `appsettings.Development.json` locally and environment variables in production.

4. Run the API

```bash
cd WatchmanAttendance.API
dotnet restore
dotnet run
```

API runs at:

- HTTP: `http://localhost:5036`
- Swagger: `http://localhost:5036/swagger`

5. Configure the frontend

Edit `WatchmanAttendance.UI/src/app/config/app-config.ts` :

```typescript
export const AppConfig = {
  apiUrl: 'http://localhost:5036/api',
  baseUrl: 'http://localhost:5036',
};
```

6. Run the frontend

```
cd WatchmanAttendance.UI
npm install
npx ng serve
```

Frontend runs at `http://localhost:4200`.

Build

Backend

```
cd WatchmanAttendance.API
dotnet build -c Release
```

Frontend

```
cd WatchmanAttendance.UI
npm run build
```

Output: `WatchmanAttendance.UI/dist/watchman-attendance-ui/browser/`

Deployment

Recommended production architecture:

```
Browser
  │
  ├── https://app.yourdomain.com  ──►  Cloudflare Pages  (Angular UI)
  │
  └── https://api.yourdomain.com  ──►  Cloudflare Worker (optional proxy)
                                            │
                                            ▼
                                     Azure App Service  (.NET API)
                                            │
                                            ▼
                                       Azure SQL Database
```

Backend — Azure App Service

1. Publish the API

```
cd WatchmanAttendance.API
dotnet publish -c Release -o ./publish
```

2. Create Azure resources

- App Service Plan (Linux or Windows, .NET 8 runtime)
- Web App
- Azure SQL Database (or connect to an existing SQL Server)

3. Set App Service configuration

Add these as **Application Settings** in the Azure Portal (or via Azure CLI):

Setting	Value
`ConnectionStrings__Default`	Your Azure SQL connection string
`Jwt__Key`	A strong random secret (32+ characters)
`Jwt__Issuer`	`WatchmanAttendanceAPI`
`Jwt__Audience`	`WatchmanAttendanceClient`
`Frontend__BaseUrl`	Your production frontend URL

4. Update CORS

In `WatchmanAttendance.API/Program.cs` , add your production frontend domain to `WithOrigins(...)` :

```
policy.WithOrigins(
    "http://localhost:4200",
    "https://your-frontend.pages.dev",
    "https://app.yourdomain.com"
)
```

5. Deploy

Deploy the `publish/` folder using any of:

- Azure CLI: `az webapp deploy`
- Visual Studio publish profile
- GitHub Actions (zip deploy)

Frontend — Cloudflare Pages

1. Update the API URL

In `WatchmanAttendance.UI/src/app/config/app-config.ts` :

```
export const AppConfig = {
  apiUrl: 'https://your-api.azurewebsites.net/api',
  baseUrl: 'https://your-api.azurewebsites.net',
};
```

2. Build

```
cd WatchmanAttendance.UI
npm install
npm run build
```

3. Create a Cloudflare Pages project

In the Cloudflare dashboard, create a new Pages project with:

Setting	Value
Framework preset	None (or Angular)
Build command	`npm run build`
Build output directory	`dist/watchman-attendance-ui/browser`

4. Fix Angular client-side routing

Add a `_redirects` file inside `WatchmanAttendance.UI/src/` (and make sure it gets copied to the build output, or add it directly to `dist/…/browser/` after build):

```
/*    /index.html    200
```

This prevents `/login`, `/scan/:token`, and other Angular routes from returning 404 on page refresh.

Alternatively, enable the "Single Page Application" fallback in the Cloudflare Pages settings.

Optional — Cloudflare Worker Proxy

Use a Cloudflare Worker to expose the Azure API under a custom domain (e.g. `api.yourdomain.com`).

Example Worker script:

```
export default {
  async fetch(request) {
    const url = new URL(request.url);
    url.hostname = 'your-api.azurewebsites.net';
    return fetch(new Request(url.toString(), request));
  }
};
```

Then update `app-config.ts`:

```
export const AppConfig = {
  apiUrl: 'https://api.yourdomain.com/api',
  baseUrl: 'https://api.yourdomain.com',
};
```

> Cloudflare Workers cannot host an ASP.NET Core app directly. Use Workers as a proxy in front of your Azure App Service only.

Frontend — Azure Static Web Apps (alternative)

If you prefer to stay entirely on Azure:

1. Create an **Azure Static Web Apps** resource.
2. Set the build output to `dist/watchman-attendance-ui/browser`.
3. Azure Static Web Apps handles SPA routing automatically via its built-in fallback.

Production Checklist

- [ ] Replace the default JWT secret with a strong random value
- [ ] Change the default admin password (`admin@surge.com` / `12345678`) immediately
- [ ] Store all secrets in Azure App Settings, not in source code
- [ ] Add your production frontend domain to CORS `WithOrigins(...)`
- [ ] Point `app-config.ts` to the production API URL
- [ ] Restrict your Google Maps API key to your production domain
- [ ] Add Google Maps Api Url in /WatchmanAttendance.UI/src/index.html
- [ ] Verify SQL Server is accessible from Azure App Service
- [ ] Deploy frontend over **HTTPS** (required for camera and GPS on mobile)
- [ ] Deploy backend over **HTTPS**
- [ ] Test the full QR scan → login → attendance flow on a real mobile device
- [ ] Configure SPA fallback (`_redirects` or platform setting) for Angular routing
- [ ] Ensure the `Uploads/` folder (or equivalent) is on persistent storage

Troubleshooting

QR opens but asks for login

Expected. If the watchman is not signed in, the app redirects to login and then continues to the attendance flow automatically after login.

Camera or GPS not working in production

Both APIs require HTTPS. Make sure your frontend and backend are both deployed with valid TLS certificates.

Angular route returns 404 on page refresh

You need a SPA fallback. Add a `_redirects` file on Cloudflare Pages or enable the SPA fallback on Azure Static Web Apps.

API blocked by CORS

Add your deployed frontend URL to `WithOrigins(...)` in `Program.cs` and redeploy the API.

`Cloudflare Worker cannot host the .NET API`

Correct — Workers run JavaScript/WASM at the edge. Use Workers as a reverse proxy in front of your Azure App Service, not as the API host.